4.3 Database Schema Documentation

4.3.1 Database Overview

The CampusCare system uses **PostgreSQL** through **Supabase** as the main relational database. The database stores users, submitted issues, worker assignments, comments, image URLs, and issue status updates.

The database is designed to support the complete CampusCare workflow:

```
Community Member submits issue
→ Facility Manager views issue
→ Facility Manager assigns worker
→ Worker views assigned issue
→ Worker updates status
→ Worker uploads completion photo
→ Facility Manager closes issue
```

The main database tables are:

Table Name	Purpose
`users`	Stores all registered users and their roles
`issues`	Stores all submitted campus issues
`comments`	Stores comments added to issues
`issue_photos`	Stores uploaded issue/completion photo records

4.3.2 Entity-Relationship Diagram

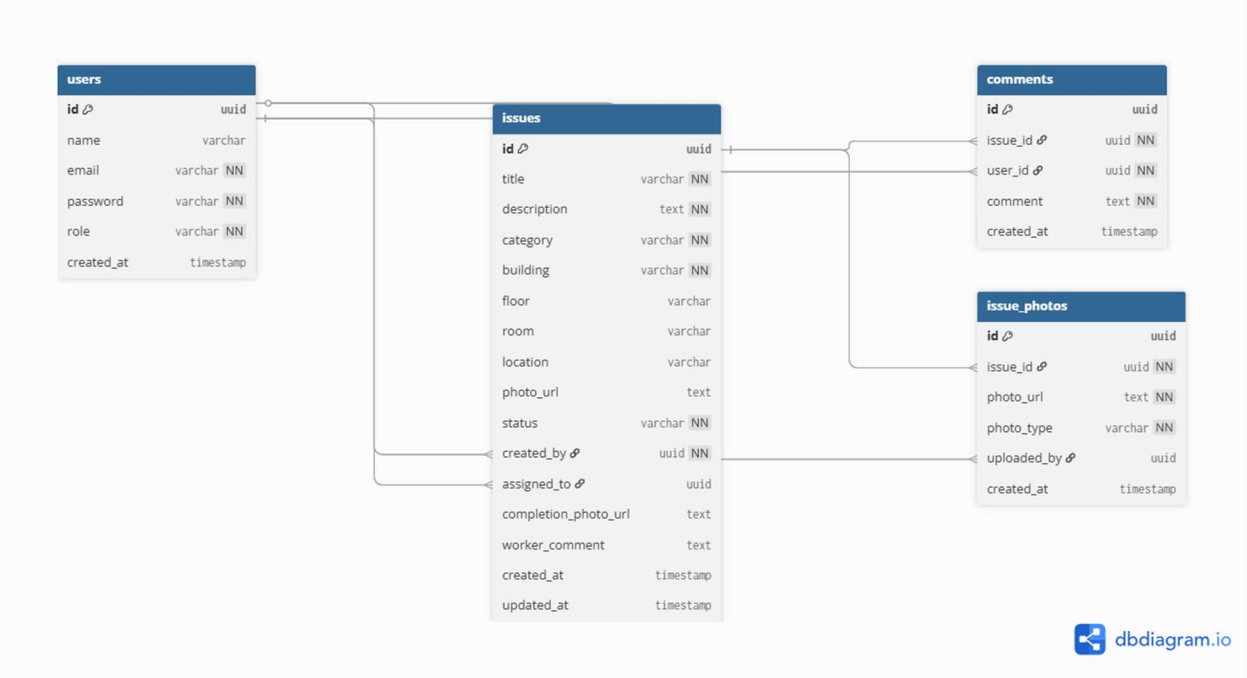

Figure 2: CampusCare Entity-Relationship Diagram

The ERD shows the relationships between the `users`, `issues`, `comments`, and `issue_photos` tables. The `issues` table is the central entity because it connects issue creators, assigned workers, comments, and uploaded photos.

4.3.3 Table Definitions

Table 1: `users`

Description

The `users` table stores all application users, including Community Members, Facility Managers, and Workers. Each user has a role that controls access to the system using Role-Based Access Control.

Columns

Column Name	Data Type	Constraints	Description
`id`	`uuid`	Primary Key	Unique identifier for each user
`name`	`varchar`	Not Null	Full name of the user
`email`	`varchar`	Unique, Not Null	User email used for login
`password_hash`	`text`	Not Null	Securely hashed password using bcrypt

Column Name	Data Type	Constraints	Description
role	text	Not Null	User role: community, manager, or worker
is_active	boolean	Default true	Indicates whether the user account is active
created_at	timestamptz	Default current timestamp	Date and time when the account was created

Primary Key

id

Notes

- The email column is unique to prevent duplicate accounts.
- The password_hash column stores encrypted passwords, not plain text.
- The role column is used for role-based navigation and backend authorization.

Table 2: issues

Description

The issues table is the core table of the CampusCare system. It stores all maintenance issues reported by Community Members. It also stores assignment, status, issue photo, completion photo, and worker comment information.

Columns

Column Name	Data Type	Constraints	Description
id	uuid	Primary Key	Unique identifier for each issue
title	text / varchar	Not Null	Short title of the issue
description	text	Not Null	Detailed description of the issue
category	text	Not Null	Issue category such as Furniture, Technical, Cleaning, etc.
building	text	Not Null	Building where the issue exists
floor	text	Nullable	Floor number/location
room	text	Nullable	Room number/location
location	text	Nullable	Combined location text if used
photo_url	text	Nullable	URL of the issue photo uploaded to cloud storage
status	text	Not Null	Current issue status

Column Name	Data Type	Constraints	Description
created_by	uuid	Foreign Key → users.id	User who submitted the issue
assigned_to	uuid	Foreign Key → users.id, Nullable	Worker assigned to the issue
completion_photo_url	text	Nullable	URL of worker completion photo
worker_comment	text	Nullable	Worker comment after resolving the issue
created_at	timestamptz	Default current timestamp	Date and time when issue was submitted
updated_at	timestamptz	Nullable / Auto-updated	Date and time when issue was last updated

Primary Key

`id`

Foreign Keys

Foreign Key	References	Meaning
created_by	users(id)	The user who submitted the issue
assigned_to	users(id)	The worker assigned to resolve the issue

Status Values

The `status` column supports the issue lifecycle:

```
Pending → In Progress → Resolved → Closed
```

Status	Meaning
Pending	Issue submitted but not assigned yet
In Progress	Issue assigned and being handled
Resolved	Worker completed the task
Closed	Facility Manager closed the issue

Notes

- `created_by` links the issue to the Community Member who submitted it.
- `assigned_to` links the issue to the Worker responsible for fixing it.
- `photo_url` stores the original issue image.
- `completion_photo_url` stores the proof-of-completion image uploaded by the worker.

Table 3: `comments`

Description

The `comments` table stores comments added to issues. It allows workers or users to record updates related to an issue.

Columns

Column Name	Data Type	Constraints	Description
`id`	`uuid`	Primary Key	Unique identifier for each comment
`issue_id`	`uuid`	Foreign Key → `issues.id`, Not Null	Issue related to the comment
`user_id`	`uuid`	Foreign Key → `users.id`, Not Null	User who added the comment
`comment_text`	`text`	Not Null	Comment content
`created_at`	`timestamptz`	Default current timestamp	Date and time when the comment was created

Primary Key

`id`

Foreign Keys

Foreign Key	References	Meaning
`issue_id`	`issues(id)`	The issue that the comment belongs to
`user_id`	`users(id)`	The user who wrote the comment

Notes

- One issue can have multiple comments.
- One user can write multiple comments.
- This table supports future expansion for issue discussions and maintenance history.

Table 4: `issue_photos`

Description

The `issue_photos` table stores metadata for uploaded photos related to issues. It supports storing multiple photos for one issue, including original issue photos and completion photos.

Columns

Column Name	Data Type	Constraints	Description
`id`	`uuid`	Primary Key	Unique identifier for each uploaded photo
`issue_id`	`uuid`	Foreign Key → `issues.id`, Not Null	Related issue
`photo_url`	`text`	Not Null	Cloud URL of the uploaded photo
`photo_type`	`text`	Not Null	Type of photo, such as `issue` or `completion`
`uploaded_by`	`uuid`	Foreign Key → `users.id`, Nullable	User who uploaded the photo
`created_at`	`timestamptz`	Default current timestamp	Date and time when the photo was uploaded

Primary Key

`id`

Foreign Keys

Foreign Key	References	Meaning
`issue_id`	`issues(id)`	The issue linked to the uploaded photo
`uploaded_by`	`users(id)`	The user who uploaded the photo

Notes

- The system currently stores main issue and completion photo URLs in the `issues` table.
- The `issue_photos` table supports future scalability by allowing multiple photos per issue.

4.3.4 Database Relationships

Relationship 1: `users` to `issues` through `created_by`

`users.id → issues.created_by`

Explanation

One Community Member can submit many issues. Each issue is created by one user.

Cardinality

```
One user → Many issues
```

Relationship 2: `users` to `issues` through `assigned_to`

```
users.id → issues.assigned_to
```

Explanation

One Worker can be assigned many issues. Each issue can be assigned to one worker.

Cardinality

```
One worker → Many assigned issues
```

Relationship 3: `issues` to `comments`

```
issues.id → comments.issue_id
```

Explanation

One issue can have many comments. Each comment belongs to one issue.

Cardinality

```
One issue → Many comments
```

Relationship 4: `users` to `comments`

```
users.id → comments.user_id
```

Explanation

One user can add many comments. Each comment is written by one user.

Cardinality

```
One user → Many comments
```

Relationship 5: `issues` to `issue_photos`

```
issues.id → issue_photos.issue_id
```

Explanation

One issue can have many uploaded photos. Each photo belongs to one issue.

Cardinality

```
One issue → Many photos
```

Relationship 6: `users` to `issue_photos`

```
users.id → issue_photos.uploaded_by
```

Explanation

One user can upload many photos. Each uploaded photo is linked to the user who uploaded it.

Cardinality

```
One user → Many uploaded photos
```

4.3.5 Database Design Justification

The database design follows relational database principles and separates system data into logical tables.

The `users` table stores authentication and role information. This supports secure login and role-based access control.

The `issues` table is the main transactional table because it stores the complete issue lifecycle, including submission details, status, assigned worker, original issue photo, completion photo, and worker comments.

The `comments` table separates issue comments from the main issue record. This improves normalization and allows future expansion where multiple comments can be stored for each issue.

The `issue_photos` table supports scalable image management by allowing multiple uploaded photos to be linked to one issue.

This design supports the current application requirements while allowing future expansion, such as multiple photos per issue, advanced discussion threads, analytics dashboards, and audit history.

4.3.6 Sample / Seed Data Used for Testing

The following sample data was used during testing to verify authentication, issue submission, worker assignment, and completion photo upload.

Sample Users

Name	Email	Role	Purpose
Test User	testuser@gmail.com	community	Used to submit issues
Manager Test	manager1@gmail.com	manager	Used to assign workers and update statuses
Worker test	worker1@giu.edu.eg	worker	Used to receive assigned issues and upload completion photos
Worker Test	worker1@gmail.com	worker	Additional worker account used to test dynamic worker selection

Sample Issues

Title	Category	Building	Floor	Room	Status	Purpose
Broken Chair	Furniture	B1	2	202	In Progress	Test issue submission and assignment
laptop	Technical	B3	4	210	Resolved	Test worker resolution workflow
Table	Furniture	F1	6	208	In Progress	Test worker dashboard assignment
board	Utensils	G1	2	307	In Progress	Test manager assignment workflow
bacha	Banzz	Ban s	5	254	In Progress	Test issue visibility and assignment

Sample Comments

Comment Text	Purpose
Table is broken	Testing issue comments
done	Testing worker update/comment flow
Fixed successfully by worker	Testing completion photo upload workflow
Work is done	Testing resolved issue workflow

Sample Uploaded Photos

Photo Type	Storage Location	Purpose
Issue Photo	Supabase Storage	Uploaded when community member submits issue
Completion Photo	Supabase Storage	Uploaded by worker after resolving issue

4.3.7 Database Security Notes

The database supports security through the following mechanisms:

- User passwords are stored as hashed values using bcrypt.
- User roles are stored in the `users.role` column.
- Backend middleware checks the user role before allowing protected operations.
- Users cannot access protected data without a valid JWT token.
- Facility Manager operations such as assignment and closing issues are protected by role-based access.
- Worker operations are limited to assigned issue workflows.

4.3.8 Database Summary

The CampusCare database schema supports the full system workflow from issue creation to issue resolution. It provides a clear relationship between users, issues, comments, and photos.

The schema is suitable for the current Milestone 2 implementation and can be expanded in the future to support additional features such as notifications, analytics, admin dashboards, and multi-campus maintenance management.